Shenzhen Hiwonder Technology Co,Ltd

Hiwonder

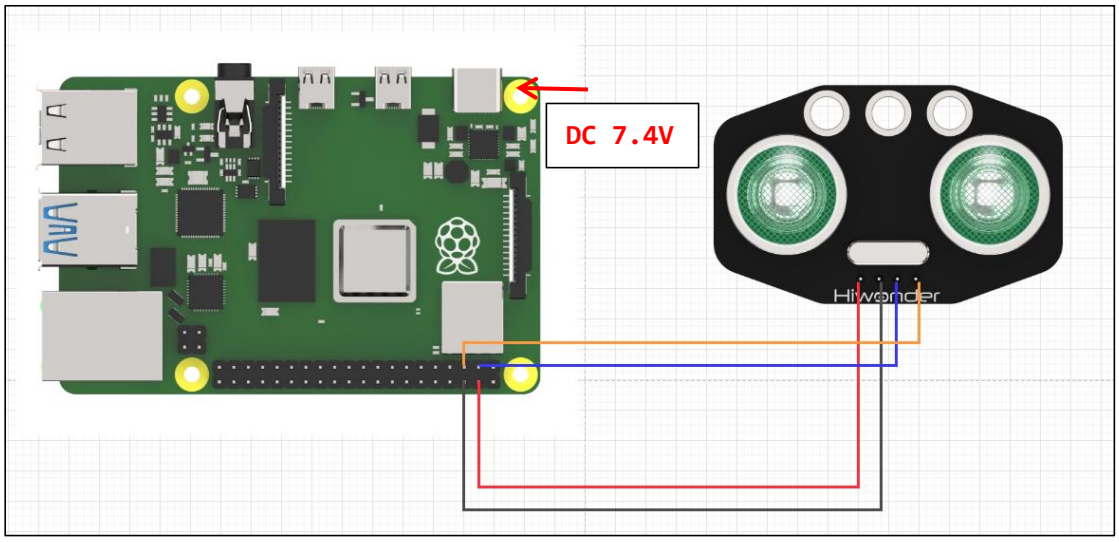
Raspberry Pi Development

V1.0

1. Getting Ready

1.1 Wiring Instruction

When wiring, the 5V, GND, SDA and SCL pins of the glowing ultrasonic module should be connected to the corresponding pins of the Raspberry Pi.



Note:

① If using Hiwonder lithium battery, please connect the red wire to the “+” terminal and the black wire to the “-” terminal of the DC connector.

② If the connector cable is not connected to the lithium battery, do not directly connect it to the battery connector cable to avoid a short circuit between the positive and negative terminals.

③ Before powering on, make sure that no metal objects contact with the board. Otherwise, there may be a risk of short circuit and damage to the board due to the pins at the bottom of the board.

1.2 Environment Configuration

Install “NoMachine” into computer, where the software package is located in “2. Software Tools & Program Collection\01 Installation Package of Each

Softwar\Remote Desk Connection Tool". You can learn detailed usage instruction of NoMachine in the corresponding directory.

Drag the program into the Raspberry Pi system. Please ensure that library files should be placed in the same directory as the program. Let's use placing it on the desktop as an example.

Open the terminal and type the command to switch the program where the program is located: "sudo chmod a+x Sonar.py"

```
ubuntu@ubuntu:~/Desktop$ sudo chmod a+x Sonar.py
```

2. Case Test

The outcome of this program is that the glowing ultrasonic module will detect distance and perform color transformation.

2.1 Program Running

Open the terminal and type the command to perform this case program: "python3 Sonar.py"

```
ubuntu@ubuntu:~/Desktop$ python3 Sonar.py
```

2.2 Program Outcome

The glowing ultrasonic module will print the detected target distance in the terminal, while the RGB light on the ultrasonic module will correspondingly perform a gradual color transformation.

2.3 Brief Analysis of Case Program

- **Import library file and check the Python version**

```
import os
import sys
import time
import smbus
from smbus2 import SMBus, i2c_msg
```

In order to make sure the program can run successfully, import os, sys, time, and other relevant function library and check the Python version. The function of these library files are as shown in the following content:

os: a library file related to operation system. It is mainly used for interacting with the system;

sys: a part of Python interpreter. It is mainly used for interacting with the interpreter, such as running command line and obtaining parameters;

time: a function library related to time. It can be used for setting timers, calculating time differences and so on;

smbus: a library file used for communicating with I2C, providing an I2C interface.

- **Instantiate the ultrasonic library**

```
class Sonar:

    __units = {"mm":0, "cm":1}
    __dist_reg = 0

    __RGB_MODE = 2
    __RGB1_R = 3
    __RGB1_G = 4
    __RGB1_B = 5
    __RGB2_R = 6
    __RGB2_G = 7
    __RGB2_B = 8

    __RGB1_R_BREATHING_CYCLE = 9
    __RGB1_G_BREATHING_CYCLE = 10
    __RGB1_B_BREATHING_CYCLE = 11
    __RGB2_R_BREATHING_CYCLE = 12
    __RGB2_G_BREATHING_CYCLE = 13
    __RGB2_B_BREATHING_CYCLE = 14
```

Instantiate the glowing ultrasonic library. And define the attributes involved in it, such as detection distance, glowing ultrasonic RGB color, breathy light

mode.

- **Initialization function**

```
def __init__(self):
    self.i2c_addr = 0x77
    self.i2c = 1
    self.R1 = 0
    self.G1 = 0
    self.B1 = 0
    self.R2 = 0
    self.G2 = 0
    self.B2 = 0
    self.RGBMode = 0
    self.bus = SMBus(self.i2c)
```

Define the I2C address, RGB color and RGB light mode of the ultrasonic module.

- **Get detection distance function**

```
def __getattr(self, attr):
    if attr in self.__units:
        return self.__units[attr]
    if attr == "Distance":
        return self.getDistance()
    else:
        raise AttributeError('Unknow attribute : %s'%attr)
```

When the condition is met, the current detection distance of the target will be obtained through the “getDistance” function. Otherwise, an error message will be prompted.

- **Set the light mode of the luminous ultrasonic module**

```
#set the light mode. 0 means colorful light mode, 1 means breathy light mode.
def setRGBMode(self, mode):
    try:
        self.bus.write_byte_data(self.i2c_addr, self.__RGB_MODE, mode)
    except:
        print('Sensor not connected!')
#set the light color

#parameter 1: 0 means the light on the left, 1 means the light on the right

#parameter 1: the RGB ratio values of color, passed as a tuple, with a range of 0-255, in sequence for r, g, and b
def setRGB(self, index, rgb):
    start_reg = 3 if index == 1 else 6
    try:
        self.bus.write_byte_data(self.i2c_addr, start_reg, rgb[0])
        self.bus.write_byte_data(self.i2c_addr, start_reg+1, rgb[1])
        self.bus.write_byte_data(self.i2c_addr, start_reg+2, rgb[2])
    except:
        print('Sensor not connected!')
#breathy light mode

#parameter 1: 0 means the light on the left, 1 means the light on the right

#parameter 2: color channel. 0 means r, 1 means g, 2 means b

#parameter 3: color change cycle. Unit: ms
def setBreathCycle(self, index, rgb, cycle):
    start_reg = 9 if index == 1 else 12
    cycle = int(cycle / 100)
    try:
        self.bus.write_byte_data(self.i2c_addr, start_reg + rgb, cycle)
    except:
        print('Sensor not connected!')
```

```
def startSymphony(self):
    self.setRGBMode(1)
    self.setBreathCycle(1, 0, 2000)
    self.setBreathCycle(1, 1, 3300)
    self.setBreathCycle(1, 2, 4700)
    self.setBreathCycle(0, 0, 4600)
    self.setBreathCycle(0, 1, 2000)
    self.setBreathCycle(0, 2, 3400)
```

Set the RGB mode of the glowing ultrasonic module through the “setRGBMode” function; set the corresponding color of light through the “setRGB” function, where 0 and 1 represent two ultrasonic lights in the left and right. And the “setBreathyCycle” function is used for setting breathy light mode.

The function receives three parameters: one is index. 0 or 1 represents the LED light on the left or right side, another one is color channel. 0, 1 or 2 represent red, green or blue, and the last one is color change cycle in milliseconds. This represents the attempt of the main controller to write the cycle value to the device through the I2C interface. If the device is not connected, the terminal will print out an error message.

Next is the “startSymphony” function, which is used for setting the breathing effect of the glowing ultrasonic light. Calling “setRGBMode (1)” to set its RGB mode at first, then the breathing effect of the red, green, and blue lights can be set separately.

After that, call the “setBreathCycle” function to set the breathing effect of color. Three parameters respectively correspond to the left and right ultrasonic lamps, the color channels, and the value of the breathing light.

- **Get detection distance**

```
#obtain distance, unit:mm
def getDistance(self):
    dist = 5000
    try:
        msg = i2c_msg.write(self.i2c_addr, [0,])
        self.bus.i2c_rdwr(msg)
        read = i2c_msg.read(self.i2c_addr, 2)
        self.bus.i2c_rdwr(read)
        dist = int.from_bytes(bytes(list(read)), byteorder='little', signed=False)
        if dist > 5000:
            dist = 5000
    except BaseException as e:
        print('Sensor not connected!', e)
    return dist
```

Call the “getDistance” function to read the I2C pin of the ultrasonic module, get the current distance to the obstacle target, and return it.

- **Print detection distance data**

```
if __name__ == '__main__':
    s = Sonar()
    s.setRGBMode(0)
    s.setRGB(1, (35, 205, 55))
    s.setRGB(0, (235, 205, 55))
    s.startSymphony()
    while True:
        time.sleep(0.1)
        print('distance: {}(mm)'.format(s.getDistance()))
```

Within the loop, the terminal will continuously print the obstacle target data detected by the ultrasonic module.